

ПРАКТИЧЕСКАЯ РАБОТА

Средства разработки технической документации. Технологии разработки документов

Цель работы

Научиться оформлять документацию на программные средства: писать docstring по стандарту, составлять автоматически генерируемую HTML-документацию с помощью pdoc3, разрабатывать и запускать тесты в pytest, а также оформлять раздел технического задания по ГОСТ 19.201-78.

Необходимое программное обеспечение

- Python 3.10+
- Библиотека pytest — установить командой:
`pip install pytest`
- Библиотека pdoc3 — установить командой:
`pip install pdoc3`
- IDE на выбор: PyCharm, VS Code или IDLE

Задание 1. Написание docstring и реализация класса

1.1 Теоретическая часть — письменные ответы

Прочитайте материал и дайте письменные ответы на вопросы ниже. Ответы записывайте в специально отведённых строках.

Вопросы для письменного ответа

1. Что такое docstring и зачем он используется в промышленной разработке?

Ответ: _____

2. Назовите минимум 2 стиля написания docstring (например, Google Style, NumPy Style):

Ответ: _____

3. Перечислите обязательные разделы docstring для метода класса:

Ответ: _____

1.2 Практическая часть — реализация класса BankAccount

Создайте файл bank_account.py. Скопируйте шаблон ниже и заполните все методы (замените комментарии # TODO реальным кодом):

```
class BankAccount:
    """
    Модель банковского счёта клиента.

    Attributes:
        owner (str): Имя владельца счёта.
```

```
balance (float): Текущий баланс счёта в рублях.
```

Example:

```
>>> acc = BankAccount("Иванов И.И.", 5000)
>>> acc.deposit(2000)
>>> acc.balance
7000
"""
```

```
def __init__(self, owner: str, balance: float = 0):
    # TODO: инициализируйте атрибуты owner и balance
    pass
```

```
def deposit(self, amount: float) -> None:
    """
```

Пополняет счёт на указанную сумму.

Args:

amount (float): Сумма пополнения. Должна быть > 0.

Raises:

ValueError: Если сумма <= 0.

"""

TODO: реализуйте метод

pass

```
def withdraw(self, amount: float) -> None:
    """
```

Снимает деньги со счёта.

Args:

amount (float): Сумма снятия. Должна быть > 0 и <= balance.

Raises:

ValueError: Если сумма <= 0 или превышает баланс.

"""

TODO: реализуйте метод

pass

```
def get_info(self) -> str:
    """
```

Возвращает строку с информацией о счёте.

Returns:

str: Строка вида "Владелец: Иванов И.И. | Баланс: 5000 руб."

"""

TODO: реализуйте метод

pass

Требования к заданию 1

- Все методы реализованы (нет pass с TODO)
- Docstring сохранены и дополнены (раздел Example в __init__)
- Написаны аннотации типов для всех параметров и возвращаемых значений
- Исключения ValueError возбуждаются корректно с понятным сообщением

Задание 2. Написание и запуск тестов pytest

Создайте файл `test_bank_account.py` в той же папке. Полностью скопируйте код тестов ниже, изучите каждый тест и запустите их командой:

Код тестов — файл `test_bank_account.py`:

```
import pytest
from bank_account import BankAccount

# == Тесты инициализации =====

def test_create_account_with_balance():
    """Счёт создаётся с правильным владельцем и балансом."""
    acc = BankAccount("Петров П.П.", 10000)
    assert acc.owner == "Петров П.П."
    assert acc.balance == 10000

def test_create_account_default_balance():
    """При создании без баланса — баланс равен 0."""
    acc = BankAccount("Сидоров С.С.")
    assert acc.balance == 0

# == Тесты пополнения =====

def test_deposit_increases_balance():
    """Пополнение увеличивает баланс на правильную сумму."""
    acc = BankAccount("Тест", 1000)
    acc.deposit(500)
    assert acc.balance == 1500

def test_deposit_zero_raises_error():
    """Пополнение на 0 вызывает ValueError."""
    acc = BankAccount("Тест", 1000)
    with pytest.raises(ValueError):
        acc.deposit(0)

def test_deposit_negative_raises_error():
    """Пополнение на отрицательную сумму вызывает ValueError."""
    acc = BankAccount("Тест", 1000)
    with pytest.raises(ValueError):
        acc.deposit(-100)

# == Тесты снятия =====

def test_withdraw_decreases_balance():
    """Снятие уменьшает баланс на правильную сумму."""
    acc = BankAccount("Тест", 2000)
    acc.withdraw(800)
```

```

assert acc.balance == 1200

def test_withdraw_full_balance():
    """Можно снять весь баланс."""
    acc = BankAccount("Тест", 500)
    acc.withdraw(500)
    assert acc.balance == 0

def test_withdraw_more_than_balance_raises_error():
    """Снятие суммы больше баланса вызывает ValueError."""
    acc = BankAccount("Тест", 300)
    with pytest.raises(ValueError):
        acc.withdraw(500)

def test_withdraw_zero_raises_error():
    """Снятие 0 вызывает ValueError."""
    acc = BankAccount("Тест", 1000)
    with pytest.raises(ValueError):
        acc.withdraw(0)

# == Тест метода get_info =====

def test_get_info_format():
    """get_info возвращает строку в правильном формате."""
    acc = BankAccount("Козлов К.К.", 7500)
    info = acc.get_info()
    assert "Козлов К.К." in info
    assert "7500" in info

```

Что записать по результатам запуска тестов

Количество пройденных тестов (PASSED): _____

Количество упавших тестов (FAILED): _____

Если есть FAILED — укажите название теста и причину ошибки:

Как вы исправили ошибку (если была):

Задание 3. Генерация HTML-документации с помощью pdoc3

После того как все тесты из Задания 2 прошли успешно, выполните команду для автоматической генерации документации из docstring:

```
pdoc --html bank_account.py --output-dir docs
```

Откройте файл docs/bank_account.html в браузере и убедитесь, что все docstring отображаются корректно.

Письменные ответы по Заданию 3

1. Какую информацию `rdoc3` автоматически берёт из вашего кода?

Ответ: _____

2. Чем автогенерация документации удобнее ручного написания документов Word?

Ответ: _____

3. Какие ещё инструменты генерации документации для Python вы знаете или нашли?

Ответ: _____

Задание 4. Составление раздела Технического задания

На основе реализованного класса `BankAccount` оформите раздел 4.1 «Требования к функциональным характеристикам» технического задания по ГОСТ 19.201-78. Заполните все пропуски, опираясь на код.

Техническое задание (фрагмент) — заполните пропуски

Наименование программного модуля: **BankAccount**

Версия: 1.0 Дата: _____

4.1 Требования к функциональным характеристикам

Программный модуль `BankAccount` должен обеспечивать:

1. Создание счёта с указанием владельца и начального баланса.

Начальный баланс по умолчанию равен _____. Тип данных `owner`: _____.

2. Пополнение счёта (метод `deposit`):

— допустимые значения суммы: _____;

— при нарушении условий программа должна: _____.

3. Снятие средств со счёта (метод `withdraw`):

— допустимые значения суммы: _____;

— условие проверки баланса: _____;

— при нарушении условий программа должна: _____.

4. Вывод информации о счёте (метод `get_info`):

Метод возвращает строку в формате: "_____".

4.2 Требования к надёжности:

— Модуль должен корректно обрабатывать некорректный ввод без аварийного завершения.

— Все исключительные ситуации должны сопровождаться: _____.

Структура сдаваемых файлов

Создайте папку с именем `Фамилия_Практика1` и поместите в неё следующие файлы:

```
Фамилия_Практика1/
├── bank_account.py      <- реализация класса с docstring
├── test_bank_account.py <- файл с тестами
├── docs/
│   └── bank_account.html <- сгенерированная документация
└── ТЗ_раздел_4.1.docx  <- оформленное техническое задание
```